

Exchange Provider Public API Documentation

Yescor

15. 12. 2025

Contents

Seyfor Yescor Exchange Provider Overview	3
First steps	3
API Authorization	3
API URLs	4
Demo environment	4
Production environment	4
API methods	4
Available endpoints for each entity	4
Recommended workflow	4
Issued invoices	4
Received invoices	6
Payment information	6
Rejection information	6
Sample requests	7
Sample request to obtain changes	7
Sample request to get a document by ID	7
Sample request to get document blob (file)	7
Document acknowledge	7
Request	7
Parameters	8
Response	8
Document changes	8
Request	8
Response	8
Document	9
Request	9
Data Properties	9
Response	9
Data Properties	10
Document submit	10
Basic Data	10
Request	10
Data Properties	11
Response	11
Document Blob	12
Request	12
Data Properties	12
Response	12
Data Properties	12
Document payment acknowledge	12
Request	12
Parameters	13
Response	13
Document payment changes	13
Request	13
Response	13
Document payment	14
Request	14
Data Properties	14
Response	14

Data Properties	15
Document payment submit	15
Basic Data	15
Request	15
Data Properties	16
Response	16
Document rejection acknowledge	17
Request	17
Parameters	17
Response	17
Document rejection changes	17
Request	18
Response	18
Document rejection	18
Request	18
Data Properties	18
Response	18
Data Properties	19
Document rejection submit	19
Basic Data	19
Request	19
Data Properties	20
Response	20

Seyfor Yescor Exchange Provider Overview

Integrators will test on the demo environment. Before testing, contact support (razvoj@yescor.hr) to provide you with the `client application secret`. With this client application secret and API user credentials (obtained from the Yescor application), you will be able to start sending data to the Yescor exchange provider.

When moving to production, contact support again to open the API client (and get the production client application secret) in the production environment.

Demo and production environments are completely separate, so data sent to the demo environment will not appear in production and vice versa.

First steps

- Contact Seyfor support to get access to the Exchange Provider API. Support team will provide you with the `client application secret` that you will use for authentication. Each software has its own client application secret, so make sure you do not share it between different software.
- Obtain API credentials from the Yescor application.

API Authorization

The Exchange Provider API uses client credentials authorization. To obtain an access token, send a POST request to the token endpoint with the following parameters:

```
curl --location 'https://erp.yescor.hr/app/identity/realms/demo-exchange-provider/protocol/openid-connect/' \
--header 'Content-Type: application/x-www-form-urlencoded' \
--data-urlencode 'grant_type=client_credentials' \
--data-urlencode 'client_id={API_CLIENT_ID}' \
--data-urlencode 'client_secret={API_CLIENT_SECRET}'
```

The response will contain an `access token` that you will use in the Authorization header for subsequent API requests.

API URLs

Demo environment

- Swagger: <https://demo.exchangehub.yescor.hr/swagger>
- Token: <https://erp.yescor.hr/app/identity/realms/demo-exchange-provider/protocol/openid-connect/token>

Production environment

- Swagger: <https://exchangehub.yescor.hr/swagger>
- Token: <https://erp.yescor.hr/app/identity/realms/exchange-provider/protocol/openid-connect/token>

API methods

The API enables processing of 3 entities: Document, DocumentPayment, and DocumentRejection.

Data is sent to the API using `submit` methods, while reading data and tracking changes is done via the `get`, `status`, and `changes` methods.

Available endpoints for each entity

Each entity (Document, DocumentPayment, DocumentRejection) supports the following endpoints:

- **GET** `/id` - Retrieves complete entity data by ID
- **POST** `/submit` - Submits a new entity for processing
- **GET** `/changes` - Retrieves a list of entities that have changed since last acknowledgement
- **POST** `/acknowledge` - Marks an entity as processed to prevent it from appearing in future `changes` calls

Additionally, Document entity has:

- **GET** `/id/blob` - Retrieves the file content (signed XML file) associated with a document

Recommended workflow

1. Call the `changes` method for an entity, which returns all changes (status changes, new documents, etc.)
2. For each record ID in the `changes` response, call the `get` method to retrieve complete entity data
3. For documents, call the `blob` method if you need to retrieve the file content (e.g., signed XML)
4. Call the `acknowledge` method with the entity ID and `concurrencyStamp` to mark the record as processed

Once acknowledged, the record will not appear in subsequent `changes` calls unless it changes again.

Example for Document entity:

Diagram is the same for DocumentPayment and DocumentRejection, just replace `/document` with `/document-payment` or `/document-rejection`. Note that DocumentPayment and DocumentRejection do not have the `blob` method.

Issued invoices

Endpoint path: `/api/v1/document`

Submitting an issued invoice:

Using the `POST submit` method, the issued invoice is sent to the inbox. The request must include the document as a Base64 encoded string in UBL 2.1 format. If there are no errors, the invoice is digitally signed (if not already signed) and prepared for sending to the recipient. When the invoice is accepted by the recipient's access point, it is sent for fiscalization to the tax authority. If the invoice could not be delivered after multiple attempts, `DeliveryWithoutEInvoice` is set to true. In this case, the invoice is also registered with the tax authority. The fiscalization status is used even though technically this is not fiscalization.

Tracking changes:

1. Use the `changes` method to check which invoices have had data changes. The method returns a list of IDs of changed invoices
2. Use the `get` method to retrieve complete invoice data, where you can check all data changes

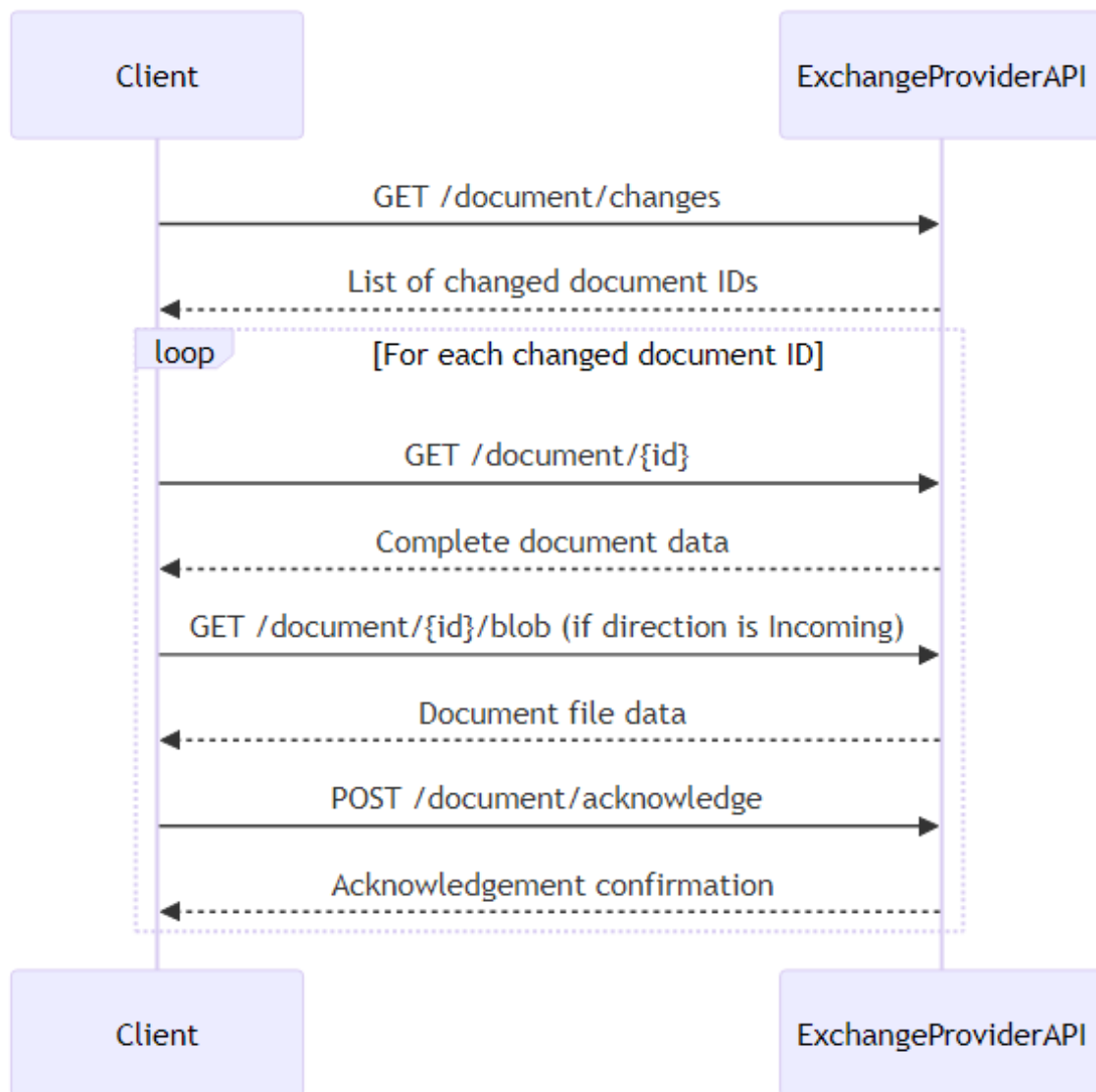


Figure 1: Diagram

3. Use the `blob` method to retrieve the invoice file (signed XML) if you need the actual file content
4. Use the `acknowledge` method to mark the invoice as processed

Processing completion criteria:

The processing of an issued invoice is complete when:

- invoice status = Sent, Error, or NotForSending and
- fiscalization status = NotRequired, Successful, or Error

Received invoices

Endpoint path: `/api/v1/document`

Tracking received invoices:

Just as with issued invoices, the `changes` method is used for received invoices to check which invoices have newly arrived in the inbox or which invoices have had data changes. The method returns a list of IDs of changed documents.

Important: The `changes` method returns a list of all documents, both issued and received. When retrieving data, you must check the `Direction` property to determine which type of invoice it is.

Each received invoice, when it arrives in the inbox, is automatically sent to the fiscalization process.

Processing workflow:

1. Use the `changes` method to get a list of changed document IDs
2. Use the `get` method to retrieve complete invoice data
3. Check the `Direction` property to distinguish between issued and received invoices
4. Use the `blob` method to retrieve the invoice file (original XML)
5. Use the `acknowledge` method to mark the invoice as processed

Processing completion criteria:

The processing of a received invoice is complete when:

- invoice status = Received or Error and
- fiscalization status = NotRequired, Successful, or Error

Payment information

Endpoint path: `/api/v1/document-payment`

Submitting payment information:

Payment information is sent using the POST `submit` method. If there are no errors, it is prepared for sending to the tax authority. The payment must reference an existing document using the document ID.

Tracking changes:

1. Use the `changes` method to check if there have been data changes
2. Use the `get` method to retrieve complete payment data
3. Use the `acknowledge` method to mark the payment as processed

Processing completion criteria:

The processing is complete when:

- status = Sent or Error

Rejection information

Endpoint path: `/api/v1/document-rejection`

Submitting rejection information:

Rejection information is sent using the POST `submit` method. If there are no errors, it is prepared for sending to the tax authority. The rejection must reference an existing document using the document ID and include rejection reason and type.

Tracking changes:

1. Use the `changes` method to check if there have been data changes
2. Use the `get` method to retrieve complete rejection data
3. Use the `acknowledge` method to mark the rejection as processed

Processing completion criteria:

The processing is complete when:

- status = Sent or Error

Sample requests

Requests must be sent to the appropriate environment (demo or production) as described in the API URLs section. Replace `{ACCESS_TOKEN}` with the token obtained from the authorization step and `{APP-SECRET}` with the client application secret provided by Seyfor support.

Sample request to obtain changes

```
curl -X 'GET' \
  'https://demo.exchangehub.yescor.hr/api/v1/document/changes' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}' \
  -H 'X-ClientApplication-Secret: {APP-SECRET}'
```

Sample request to get a document by ID

```
curl -X 'GET' \
  'https://demo.exchangehub.yescor.hr/api/v1/document/{id}' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}' \
  -H 'X-ClientApplication-Secret: {APP-SECRET}'
```

Sample request to get document blob (file)

```
curl -X 'GET' \
  'https://demo.exchangehub.yescor.hr/api/v1/document/{id}/blob' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer {ACCESS_TOKEN}' \
  -H 'X-ClientApplication-Secret: {APP-SECRET}'
```

Document acknowledge

Marking the status of a transferred document.

The `document/acknowledge` request marks that the latest changes to the document have been successfully transferred to the ERP.

Request

POST `/api/v{version}/document/acknowledge`

Headers:

Authorization: Bearer `{{Account access token}}`

X-ClientApplication-Secret: `{{Client application secret}}`

```
{
  "id": "3a11d48b-b33d-74fe-f8de-fa6aff7ae4d9",
  "concurrencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
}
```

Parameters

- id (guid): Unique document identifier
- concurrencyStamp (string): For preventing conflicts during save operations

Response

Response HttpStatusCode: 200 OK

```
{
  "validationErrors": []
}
```

Response HttpStatusCode: 400 NOT OK

```
{
  "error": {
    "code": null,
    "message": "Your request is not valid!",
    "details": "During validation, the following errors were detected.\n - The item could not be found",
    "data": {},
    "validationErrors": [
      {
        "message": "[[Error]]::[[ExchProvExchProv:Document.Error.ConcurrencyConflict]]::The item c",
        "members": []
      }
    ]
  }
}
```

Document changes

Retrieving a list of document IDs whose latest changes have not been transferred to the ERP.

Request

GET /api/v{version}/document/changes

Headers:

Authorization: Bearer {{Account access token}}

X-ClientApplication-Secret: {{Client application secret}}

Retrieves the list of documents not yet transferred.

Response

Response HttpStatusCode: 200 OK

```
{
  "data": {
    "items": [
      "59a33050-dab8-4c92-baa7-538d5e876631",
      "d0732166-29d6-495d-be40-b558e96bdf67",
      "0fb53237-e00a-47b3-8b68-f6240c7938db"
    ],
    "totalCount": 3
  }
}
```

```

    },
    "validationErrors": []
}

• items: List of document IDs

```

Document

Request

```

GET /api/v{version}/document/{id}
Headers:
Authorization: Bearer {{Account access token}}
X-ClientApplication-Secret: {{Client application secret}}

```

Data Properties

- id (guid): Unique identifier of the document whose data is being retrieved.

If there are no errors, retrieve document data with data:

- tenantId (obtained based on Account access token)
- id

Response

Response HttpStatusCode: 200 OK

```

{
  "data": {
    "id": "b1293eee-dd22-4a93-92dd-28927a72faee",
    "documentType": 1,
    "direction": 1,
    "creationDateTimeUtc": "2025-11-24T12:13:02.037383Z",
    "status": 3,
    "processedDateTimeUtc": "2025-11-24T12:13:33.45879Z",
    "errorMessage": null,
    "fiscalizationStatus": 3,
    "FiscalizationDateTimeUtc": "2025-11-24T12:12:33.45879Z",
    "fiscalizationErrorMessage": null,
    "partyTaxNumber": "123456789",
    "partyName": "Naziv subjekta d.o.o.",
    "documentNumber": "INV-2025-0001",
    "documentDate": "2025-09-13",
    "deliveryWithoutEInvoice": false,
    "isSynchronized": true,
    "lastSynchronizedAtUtc": "2025-11-25T11:24:33.45879Z",
    "currencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
  },
  "validationErrors": []
}

```

Response HttpStatusCode: 404 NOT OK

```

{
  "error": {
    "code": null,
    "message": "There is no Document_ReadAggregate entity with id = 3a1dc7f4-f4d5-7daa-8c16-296af0c066",
    "details": null,
    "data": null,
    "validationErrors": null
  }
}

```

```
}  
}
```

Data Properties

- **id** (guid): Unique document identifier
- **documentType** (string): Document type
 - (1) Invoice
 - (2) CreditNote
- **direction** (int): Document direction
 - (1) Outgoing
 - (2) Incoming
- **creationDateTime** (dateTime): Document creation date and time
- **status** (int): Document status
 - (1) New (document has been taken over by the provider, awaiting validation and document signing)
 - (2) Sending (document is in the process of being sent to the recipient)
 - (3) Sent (document has been successfully sent to the recipient)
 - (4) Error (an error occurred during processing or sending the document)
 - (5) Received (document has been successfully received in our inbox)
 - (6) NotForSending (document is not intended for sending - Delivery without e-invoice issuance)
- **processedDateTimeUtc** (dateTime): Date and time of completion of document sending processing
- **errorMessage** (string): Error message if an error occurred
- **fiscalizationStatus** (int): Fiscalization status
 - (1) NotRequired (document is not subject to fiscalization)
 - (2) Pending (document is waiting to start the process or is in the fiscalization process)
 - (3) Successful (document is fiscalized)
 - (4) Error (an error occurred during the fiscalization process)
- **FiscalizationDateTimeUtc** (dateTime): Date and time of completion of fiscalization processing
- **fiscalizationErrorMessage** (string): Fiscalization error message if an error occurred
- **partyTaxNumber** (string): Tax number of the entity
- **partyName** (string): Name of the entity
- **documentNumber** (string): Document number
- **documentDate** (date): Document date
- **deliveryWithoutEInvoice** (boolean): Delivery without e-invoice issuance
- **isSynchronized** (boolean): Whether the document is synchronized with an external system
- **lastSynchronizedAtUtc** (dateTime): Date and time of last synchronization of the document with an external system
- **concurrencyStamp** (string): Stamp for detecting concurrent changes

Document submit

Uploading a document for sending to the recipient and fiscalization.

Basic Data

- Data structure & validations

Request

```
POST /api/v{version}/document/submit  
Headers:  
Authorization: Bearer {{Account access token}}  
X-ClientApplication-Secret: {{Client application secret}}  
  
{  
  "documentType": 1,  
  "mainFile": {
```

```

        "fileName": "invoice_12345.xml",
        "mimeType": "text/xml",
        "data": "JVBERi0xLjQNCiX5+prnDQo0IDAgb2JqDQo8PAovRSAONjg1NDYKL0ggWzM2MDAgMzExXQovTCAONjg4NzQKL0xpbw
    },
    "deliveryWithoutEInvoice": false
}

```

Data Properties

- **documentType** (int): Document type.
 - (1) Invoice
 - (2) CreditNote
- **mainFile** data of the document being sent
 - **data** (DocumentBase64): document content (XML file in UBL 2.1 format)
 - **fileName** (SingleLineString): file name with appropriate extension (.xml)
 - **mimeType** (SingleLineString): MIME type of the file (e.g. application/xml)
- **deliveryWithoutEInvoice** (boolean): whether this is a delivery without e-invoice issuance (true/false). If set to true, the document will not be sent to the recipient and the delivery will be recorded as delivery without e-invoice.

Response

If Tenant.SMLStatus ≠ Registered, an error is returned:

- {Tenant.Error.SMPNotRegistered}Inbox is not registered in the document exchange system.

Response HttpStatusCode: 200 OK

```

{
    "data": "9846422c-4c04-4819-92f0-bcbe3f7ecf90",
    "validationErrors": []
}

```

Response HttpStatusCode: 400 NOT OK

```

{
    "error": {
        "code": null,
        "message": "Your request is not valid!",
        "details": "During validation, the following errors were detected.\n - The Data field is required.",
        "data": {},
        "validationErrors": [
            {
                "message": "[[Error]]::[[ ]]:The Data field is required.",
                "members": [
                    "MainFile.Data::Data:"
                ]
            },
            {
                "message": "[[Error]]::[[ ]]:The FileName field is required.",
                "members": [
                    "MainFile.FileName::FileName:"
                ]
            }
        ]
    }
}

```

Document Blob

Request

```
GET /api/v{version}/document/{id}/blob
Headers:
Authorization: Bearer {{Account access token}}
X-ClientApplication-Secret: {{Client application secret}}
```

Data Properties

- `id` (guid): Unique identifier of the document whose data is being retrieved.

If there are no errors, retrieve document file data with data:

- `tenantId` (obtained based on Account access token)
- `id`

Response

Response HttpStatusCode: 200 OK

```
{
  "data": {
    "fileName": "invoice-2025-0001.xml",
    "mimeType": "text/xml",
    "data": "JVBERi0xLjQNCiX5+prnDQo0IDAgb2JqDQo8PAovRSAONjg1NDYKL0ggWzM2MDAgMzExXQovTCAONjg4NzQKL0xpbw
  },
  "validationErrors": []
}
```

Response HttpStatusCode: 404 NOT OK

```
{
  "error": {
    "code": null,
    "message": "There is no DocumentBlob_ReadAggregate entity with id = 3a1dc7f4-f4d5-7daa-8c16-296af0
    "details": null,
    "data": null,
    "validationErrors": null
  }
}
```

Data Properties

- `fileName` (string): Name of the main document file
- `mimeType` (string): MIME type of the main document file
- `data` (string): Content of the main document file in Base64 encoded format

Document payment acknowledge

Marking the status of document payment changes transfer.

The `document-payment/acknowledge` request marks that the latest changes to document payments have been successfully transferred to the ERP.

Request

```
POST /api/v{version}/document-payment/acknowledge
Headers:
Authorization: Bearer {{Account access token}}
```

```
X-ClientApplication-Secret: {{Client application secret}}
{
  "id": "3a11d48b-b33d-74fe-f8de-fa6aff7ae4d9",
  "concurrencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
}
```

Parameters

- id (guid): Unique document payment identifier
- concurrencyStamp (string): For preventing conflicts during save operations

Response

Response HttpStatusCode: 200 OK

```
{
  "validationErrors": []
}
```

Response HttpStatusCode: 400 NOT OK

```
{
  "error": {
    "code": null,
    "message": "Your request is not valid!",
    "details": "During validation, the following errors were detected.\n - The item could not be found",
    "data": {},
    "validationErrors": [
      {
        "message": "[[Error]]::[[ExchProvExchProv:DocumentPayment.Error.ConcurrencyConflict]]::The",
        "members": []
      }
    ]
  }
}
```

Document payment changes

Retrieving a list of document payment IDs whose latest changes have not been transferred to the ERP.

Request

GET /api/v{version}/document-payment/changes

Headers:

Authorization: Bearer {{Account access token}}

X-ClientApplication-Secret: {{Client application secret}}

Retrieves the list of document payments not yet transferred.

Response

Response HttpStatusCode: 200 OK

```
{
  "data": {
    "items": [
      "3a1dd77b-f0ac-5ba4-fd14-5dfb3f71e25b",
      "3a1dd77e-8f16-c71f-95e7-82e3cc87bf6c",
      "3a1dd77e-a8fc-5e10-58f1-866fa77e752b",
      "3a1dd781-d33f-f69f-b0af-4064bfb2805d",
    ]
  }
}
```

```

        "3a1dd782-520f-16a0-e854-ca84535c28ce",
        "3a1dd782-a420-8290-a6a4-acf0c1e09592",
        "3a1dd782-a72e-edfb-bd08-b32ae408fce2",
        "3a1dd782-c161-1ad1-7142-4e4d203ca767",
        "3a1dd78f-a158-9cdd-7068-9ee6b8c8f1d6",
        "3a1dd78f-c52a-01a5-3cf2-75d7c1b4675a",
        "3a1ddd13-7493-a7ac-9c9b-063f65a23950"
    ],
    "totalCount": 11
  },
  "validationErrors": []
}

```

- items: List of document IDs

Document payment

Request

```

GET /api/v{version}/document-payment/{id}
Headers:
Authorization: Bearer {{Account access token}}
X-ClientApplication-Secret: {{Client application secret}}

```

Data Properties

- id (guid): Unique document payment identifier.

If there are no errors, retrieve document payment data with data:

- tenantId (obtained based on Account access token) and
- id

Response

```

Response HttpStatusCode: 200 OK
{
  "data": {
    "id": "b1293eee-dd22-4a93-92dd-28927a72faee",
    "documentId": "4758622c-4ca2-1143-92f0-bcbe3f3e1f42",
    "documentNumber": "2025-001000254",
    "documentDate": "2025-01-15",
    "documentPartyName": "Podjetje d.o.o.",
    "documentPartyTaxNumber": "12345678901",
    "paymentDate": "2025-01-20",
    "amount": 1250.50,
    "paymentType": 1,
    "status": 3,
    "processedDateTimeUtc": "2025-11-27T12:35:16.631682Z",
    "errorMessage": null,
    "isSynchronized": true,
    "lastSynchronizedAtUtc": "2025-12-03T11:24:33.45879Z",
    "currencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
  },
  "validationErrors": []
}

```

```

Response HttpStatusCode: 404 NOT OK
{

```

```

    "error": {
      "code": null,
      "message": "There is no DocumentPayment_Aggregate entity with id = 3a1dd77b-f0ac-5ba4-fd14-5dfb3f7",
      "details": null,
      "data": null,
      "validationErrors": null
    }
  }
}

```

Data Properties

- **id** (guid): Unique document payment identifier
- **document** (guid): Unique document identifier, if the information exists
- **documentNumber** (string): Number of the document associated with the payment
- **documentDate** (string): Date of the document associated with the payment
- **partyName** (string): Name of the entity
- **documentPartyTaxNumber** (string): Tax number of the entity
- **paymentDate** (string): Payment date
- **amount** (number): Payment amount
- **paymentType** (number): Payment type
 - (1) Transaction account
 - (2) Settlement
 - (3) Other
- **status** (int): Document payment report status
 - (1) New (document payment information has been taken over by the provider)
 - (2) Sending (document payment report is in the process of being sent)
 - (3) Sent (document payment report has been successfully sent)
 - (4) Error (an error occurred during processing or sending document payment information)
- **processedDateTimeUtc** (dateTime): Date and time of request processing
- **errorMessage** (string): In case of error, contains error description
- **isSynchronized** (boolean): Whether the document is synchronized with an external system
- **lastSynchronizedAtUtc** (dateTime): Date and time of last synchronization of the document with an external system
- **concurrencyStamp** (string): Stamp for detecting concurrent changes

Document payment submit

Basic Data

- Data structure & validations

Request

POST /api/v{version}/document-payment/submit

Headers:

Authorization: Bearer {{Account access token}}

X-ClientApplication-Secret: {{Client application secret}}

```

{
  "documentId": "4758622c-4ca2-1143-92f0-bcbe3f3e1f42",
  "documentNumber": "2025-001000254",
  "documentDate": "2025-01-15",
  "documentPartyName": "Podjetje d.o.o.",
  "documentPartyTaxNumber": "12345678901",
  "paymentDate": "2025-01-20",
  "amount": 1250.50,

```

```

    "paymentType": 1
}

```

Data Properties

- `documentId` (guid): Unique document identifier, if the information exists.
- `documentNumber` (string): Number of the document associated with the payment.
- `documentDate` (string): Date of the document associated with the payment.
- `documentPartyName` (string): Name of the document recipient entity.
- `documentPartyTaxNumber` (string): Tax number of the document recipient entity.
- `paymentDate` (string): Payment date.
- `amount` (number): Payment amount.
- `paymentType` (int): Payment type.
 - (1) Transaction account
 - (2) Settlement
 - (3) Other

Response

If `Tenant.SMLStatus` $\neq$ Registered, an error is returned:

- `{Tenant.Error.SMPNotRegistered}` Inbox is not registered in the document exchange system.

Response HttpStatusCode: 200 OK

```

{
  "data": "9846422c-4c04-4819-92f0-bcbe3f7ecf90",
  "validationErrors": []
}

```

Response HttpStatusCode: 400 NOT OK

```

{
  "error": {
    "code": null,
    "message": "Your request is not valid!",
    "details": "During validation, the following errors were detected.\n - The DocumentNumber field is",
    "data": {},
    "validationErrors": [
      {
        "message": "[[Error]]::[[ ]]:The DocumentNumber field is required.",
        "members": [
          "DocumentNumber::DocumentNumber::"
        ]
      },
      {
        "message": "[[Error]]::[[ ]]:The DocumentPartyTaxNumber field is required.",
        "members": [
          "DocumentPartyTaxNumber::DocumentPartyTaxNumber::"
        ]
      }
    ]
  }
}

```

Response HttpStatusCode: 404 NOT OK

```

{
  "error": {
    "code": null,
    "message": "There is no Document_Aggregate entity with id = 4758622c-4ca2-1143-92f0-bcbe3f3e1f42!",
    "details": null,

```

```

    "data": null,
    "validationErrors": null
  }
}

```

Document rejection acknowledge

Marking the status of document rejection changes transfer.

The document-rejection/acknowledge request marks that the latest changes to document rejection have been successfully transferred to the ERP.

Request

POST /api/v{version}/document-rejection/acknowledge
 Headers:
 Authorization: Bearer {{Account access token}}
 X-ClientApplication-Secret: {{Client application secret}}

```

{
  "id": "3a11d48b-b33d-74fe-f8de-fa6aff7ae4d9",
  "concurrencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
}

```

Parameters

- id (guid): Unique document rejection identifier
- concurrencyStamp (string): For preventing conflicts during save operations

Response

Response HttpStatusCode: 200 OK

```

{
  "validationErrors": []
}

```

Response HttpStatusCode: 400 NOT OK

```

{
  "error": {
    "code": null,
    "message": "Your request is not valid!",
    "details": "During validation, the following errors were detected.\n - The item could not be found",
    "data": {},
    "validationErrors": [
      {
        "message": "[[Error]]::[[ExchProvExchProv:DocumentRejection.Error.ConcurrencyConflict]]::T",
        "members": []
      }
    ]
  }
}

```

Document rejection changes

Retrieving a list of document rejection IDs whose latest changes have not been transferred to the ERP.

Request

```
GET /api/v{version}/document-rejection/changes
Headers:
Authorization: Bearer {{Account access token}}
X-ClientApplication-Secret: {{Client application secret}}
```

Retrieves the list of document rejections not yet transferred.

Response

Response HttpStatusCode: 200 OK

```
{
  "data": {
    "items": [
      "37db2c25-7c40-43e9-8ac6-17d76c3e1bb1",
      "359e5731-9e65-464d-a0a0-ee0bced36d6c",
      "0201bd17-8340-4caf-b414-4ee670363407"
    ],
    "totalCount": 3
  },
  "validationErrors": []
}
```

- items: List of document IDs

Document rejection

Request

```
GET /api/v{version}/document-rejection/{id}
Headers:
Authorization: Bearer {{Account access token}}
X-ClientApplication-Secret: {{Client application secret}}
```

Data Properties

- id (guid): Unique document rejection identifier.

If there are no errors, retrieve document rejection data with data:

- tenantId (obtained based on Account access token) and
- id

Response

Response HttpStatusCode: 200 OK

```
{
  "data": {
    "id": "b1293eee-dd22-4a93-92dd-28927a72faee",
    "documentId": "4758622c-4ca2-1143-92f0-bcbe3f3e1f42",
    "documentNumber": "2025-001000254",
    "documentDate": "2025-01-15",
    "documentPartyName": "Podjetje d.o.o.",
    "documentPartyTaxNumber": "12345678",
    "rejectionDate": "2025-01-20",
    "rejectionReasonType": 1,
    "rejectionReason": "Missing tax number",
    "status": 3,
  }
}
```

```

        "processedDateTimeUtc": "2025-01-16T10:30:00Z",
        "errorMessage": null,
        "isSynchronized": true,
        "lastSynchronizedAtUtc": "2025-09-13 11:24:33.45879Z",
        "concurrencyStamp": "973a26e9a70f4fbc976fc9e5f850c649"
    },
    "validationErrors": []
}

Response HttpStatusCode: 404 NOT OK
{
    "error": {
        "code": null,
        "message": "There is no DocumentRejection_ReadAggregate entity with id = 37db2c25-7c40-43e9-8ac6-1",
        "details": null,
        "data": null,
        "validationErrors": null
    }
}

```

Data Properties

- **id (guid):** Unique document payment identifier
- **documentId (guid):** Unique document identifier, if the information exists
- **documentNumber (string):** Number of the document associated with the payment
- **documentDate (string):** Date of the document associated with the payment
- **documentPartyName (string):** Name of the entity
- **documentPartyTaxNumber (string):** Tax number of the entity
- **rejectionDate (string):** Rejection date
- **rejectionReasonType (number):** Rejection reason type
 - (1) Data discrepancy that does not affect tax calculation
 - (2) Data discrepancy that affects tax calculation
 - (3) Other
- **rejectionReason (string):** Rejection reason
- **status (int):** Document payment report status
 - (1) New (document payment information has been taken over by the provider)
 - (2) Sending (document payment report is in the process of being sent)
 - (3) Sent (document payment report has been successfully sent)
 - (4) Error (an error occurred during processing or sending document payment information)
- **processedDateTimeUtc (dateTime):** Date and time of request processing
- **errorMessage (string):** In case of error, contains error description
- **isSynchronized (boolean):** Whether the document is synchronized with an external system
- **lastSynchronizedAtUtc (dateTime):** Date and time of last synchronization of the document with an external system
- **concurrencyStamp (string):** Stamp for detecting concurrent changes

Document rejection submit

Basic Data

- Data structure & validations

Request

```

POST /api/v{version}/document-rejection/submit
Headers:
Authorization: Bearer {{Account access token}}

```

X-ClientApplication-Secret: `{{Client application secret}}`

```
{
  "documentId": "4758622c-4ca2-1143-92f0-bcbe3f3e1f42",
  "documentNumber": "2025-001000254",
  "documentDate": "2025-01-15",
  "documentPartyName": "Podjetje d.o.o.",
  "documentPartyTaxNumber": "12345678",
  "rejectionDate": "2025-01-20",
  "rejectionReasonType": 1,
  "rejectionReason": "Missing tax number"
}
```

Data Properties

- `documentId` (guid): Unique document identifier, if the information exists.
- `documentNumber` (string): Number of the document associated with the payment.
- `documentDate` (string): Date of the document associated with the payment.
- `documentPartyName` (string): Name of the document issuer entity.
- `documentPartyTaxNumber` (string): Tax number of the document issuer entity.
- `rejectionDate` (string): Rejection date.
- `rejectionReasonType` (int): Rejection reason type.
 - (1) Data discrepancy that does not affect tax calculation
 - (2) Data discrepancy that affects tax calculation
 - (3) Other
- `rejectionReason` (string): Rejection reason.

Response

If `Tenant.SMLStatus` $\neq$ Registered, an error is returned:

- `{Tenant.Error.SMPNotRegistered}` Inbox is not registered in the document exchange system.

Response HttpStatusCode: 200 OK

```
{
  "data": "3a1dfc28-9501-76a6-0cde-6b4c57f4fd37",
  "validationErrors": []
}
```

Response HttpStatusCode: 400 NOT OK

```
{
  "error": {
    "code": null,
    "message": "Your request is not valid!",
    "details": "During validation, the following errors were detected.\n - The DocumentNumber field is",
    "data": {},
    "validationErrors": [
      {
        "message": "[[Error]]::[[ ]]:The DocumentNumber field is required.",
        "members": [
          "DocumentNumber::DocumentNumber::"
        ]
      },
      {
        "message": "[[Error]]::[[ ]]:The DocumentPartyTaxNumber field is required.",
        "members": [
          "DocumentPartyTaxNumber::DocumentPartyTaxNumber::"
        ]
      }
    ]
  }
}
```

```
    ]
  }
}

Response HttpStatusCode: 404 NOT FOUND
{
  "error": {
    "code": null,
    "message": "There is no Document_Aggregate entity with id = 3a1da463-a6e0-8b86-03a0-c92f2a165511!"
    "details": null,
    "data": null,
    "validationErrors": null
  }
}
```